A
Project report
On
"QR Based Visitor Management System"

BY

MAHWISH IFTKHAR B.TECH

CSE

CERTIFICATE

This is to certify that **MAHWISH IFTKHAR**, student of **B.Tech (CSE)** from **AKS University, Satna**, has successfully completed the project report titled **"QR Based Visitor Management System"** during the internship at **MPOnline Limited**.

This project report is submitted in partial fulfillment of the requirements for the completion of the internship program conducted under **MPOnline Limited**.

The work presented in this report is the original work carried out by the student under the guidance and supervision of the concerned mentor during the internship period. The project demonstrates the practical implementation of a digital visitor management system using QR technology for secure and efficient visitor entry and tracking.

Project Guide / Mentor
Name: DHRUV PANDAY

Intern / Student
Name: MAHWISH IFTKHAR

Date: 26/04/2026

ABSTRACT

The QR-Based Visitor Management System is a modern solution designed to replace traditional manual visitor logbooks with a secure, efficient, and contactless digital system. The system allows visitors to register their details online or at the entrance, after which a unique QR code is generated for identification. This QR code is scanned at the entry and exit points to record the visitor's check-in and check-out time automatically.

The system enhances security by maintaining accurate digital records of all visitors and reducing the chances of unauthorized access. It also minimizes human error, speeds up the registration process, and provides a seamless experience for both visitors and security personnel. Administrators can monitor visitor activity through a centralized dashboard, filter records, and generate reports when required.

Additionally, the system supports features such as real-time tracking, data storage, and optional notifications to hosts about visitor arrivals. Built using modern technologies, it ensures reliability, scalability, and ease of use. Overall, the QR-Based Visitor Management System offers an effective and smart approach to managing visitor data while improving safety, efficiency, and user convenience.

TABLE OF CONTENTS

• Introduction.....5.

• Objective of The Project.....6.

• Scope of The Projects.....7

• Definition of Problem.....8

• Benefit of The Projects.....9

• Main Module.....10

• Technical Overview.....11

• System Configuration….....12-13

• Block Diagram.....14-15

• DFD Diagram.....16

• ER Diagram.....17-18

• Project Explanation….....19

• Codding (Sample with explanation).....20-35

• Result and Analysis.....36-38

• Software Testing.....39-42

• Conclusion….....43

INTRODUCTION

In today's digital era, security and efficient management of visitors have become essential for organizations, institutions, and residential complexes. Traditional visitor management systems rely on manual logbooks, which are time-consuming, prone to human error, and lack proper data security. These limitations create challenges in tracking visitor records, verifying identities, and ensuring overall safety.

The QR-Based Visitor Management System is designed to overcome these issues by providing a smart, automated, and contactless solution. This system allows visitors to register their details digitally and receive a unique QR code, which serves as their identification during entry and exit. By scanning the QR code, the system records real-time data such as check-in and check-out times, ensuring accurate and secure record-keeping.

The system not only enhances security but also improves efficiency by reducing waiting time and eliminating paperwork. It provides a user-friendly interface for visitors, security personnel, and administrators. Additionally, it offers features such as centralized data storage, easy retrieval of visitor records, and monitoring through an admin dashboard.

Overall, the QR-Based Visitor Management System represents a modern approach to visitor handling by integrating technology to ensure safety, convenience, and effective management. It is a reliable solution that meets the growing need for secure and digital visitor tracking systems.

OBJECTIVE OF THE PROJECT

The primary objective of the QR-Based Visitor Management System is to develop a secure, efficient, and automated solution for managing visitor entry and exit using digital technology. The system aims to replace traditional manual methods with a contactless and reliable approach.

The key objectives of the project are as follows:

- To design a system that enables **easy and quick visitor registration** through a digital interface.
- To generate a **unique QR code** for each visitor for identification and tracking purposes.
- To implement a **contactless check-in and check-out process** using QR code scanning.
- To maintain **accurate and real-time records** of visitor entry and exit.
- To enhance **security** by reducing unauthorized access and improving monitoring.
- To minimize **human errors** associated with manual record-keeping.
- To provide an **admin dashboard** for viewing, managing, and analyzing visitor data.
- To ensure **data storage and retrieval** in a structured and efficient manner.
- To reduce waiting time and improve the overall **visitor experience**.
- To build a system that is **scalable, user-friendly, and easy to maintain**.

Overall, the project aims to deliver a modern visitor management solution that improves operational efficiency, strengthens security, and provides a seamless experience for both visitors and administrators.

SCOPE OF THE PROJECT

The scope of the QR-Based Visitor Management System defines the boundaries and functionalities that the system will cover. It focuses on providing a digital and automated solution for managing visitor entry and exit using QR code technology.

The system will include the following features and functionalities:

- **Visitor Registration:**
  Visitors can enter their details such as name, contact number, and purpose of visit through a web-based interface.
- **QR Code Generation:**
  A unique QR code will be generated for each visitor after successful registration.
- **Entry and Exit Tracking:**
  QR codes will be scanned at entry and exit points to record check-in and check-out time automatically.
- **Admin Dashboard:**
  Administrators can view, search, and manage visitor records through a centralized dashboard.
- **Data Storage:**
  All visitor data will be stored securely in a database for future reference and reporting.
- **Basic Security Features:**
  The system will ensure that each QR code is unique and linked to a specific visitor record.

Application Areas

The system can be used in:

- Offices and corporate buildings
- Schools and colleges
- Hospitals
- Residential societies
- Event venues

Future Scope

- Mobile application development
- SMS/Email notifications
- Integration with access control systems
- Cloud-based deployment for scalability

DEFINITION OF THE PROJECT

The QR-Based Visitor Management System is a digital application designed to manage and monitor visitor entry and exit in a secure, efficient, and contactless manner. It replaces traditional manual logbook systems by using QR code technology to identify and track visitors within an organization or facility.

In this system, visitors are required to register their details through a web-based interface, after which a unique QR code is generated for each individual. This QR code acts as a digital pass and is scanned at the entry and exit points to automatically record the visitor's check-in and check-out time. All the information is stored in a centralized database, which can be accessed and managed by administrators through a dashboard.

The project aims to provide a reliable solution that improves security, reduces human errors, and ensures accurate record-keeping. It also enhances operational efficiency by minimizing paperwork and speeding up the visitor handling process. Overall, the system serves as a modern and smart approach to visitor management using digital technology.

BENEFITS OF THE PROJECT

The QR-Based Visitor Management System offers several advantages over traditional manual methods by improving efficiency, security, and user experience. The key benefits of the project are as follows:

- **Enhanced Security:**
  The system maintains accurate digital records of all visitors, helping to prevent unauthorized access and improve overall safety.
- **Contactless Process:**
  QR code scanning enables a touch-free check-in and check-out process, which is faster and more hygienic.
- **Time Efficiency:**
  Reduces waiting time during visitor entry by automating registration and verification.
- **Accurate Record Keeping:**
  Eliminates errors associated with manual logbooks and ensures precise tracking of entry and exit times.
- **Easy Data Management:**
  All visitor information is stored in a structured database, making it easy to search, retrieve, and manage records.
- **Real-Time Monitoring:**
  Administrators can track visitor activity in real-time through the dashboard.
- **Paperless System:**
  Reduces the use of paper, making the system eco-friendly and cost-effective.
- **Improved Visitor Experience:**
  Provides a smooth and hassle-free process for visitors with minimal effort.
- **Centralized Control:**
  Enables administrators to manage and monitor all visitor data from a single interface.
- **Scalability:**
  The system can be easily expanded and upgraded with additional features in the future.

MAIN MODULE

The QR-Based Visitor Management System is divided into several main modules, each responsible for a specific functionality to ensure smooth operation of the system. The key modules are as follows:

1. Visitor Registration Module

This module allows visitors to enter their details such as name, contact number, and purpose of visit through a digital form. It ensures that all required information is collected before generating a QR code.

2. QR Code Generation Module

After successful registration, this module generates a unique QR code for each visitor. The QR code acts as a digital identity and is linked to the visitor's record in the database.

3. QR Code Scanning Module

This module is used by security personnel to scan the visitor's QR code at the entry and exit points. It validates the QR code and updates the visitor's status in the system.

4. Entry/Exit Management Module

It records the check-in and check-out time of visitors automatically when the QR code is scanned. This helps in maintaining accurate visit duration records.

5. Database Management Module

This module stores all visitor-related data securely, including personal details, QR code ID, and entry/exit logs. It ensures proper data organization and retrieval.

6. Admin Dashboard Module

Provides an interface for administrators to view, manage, and monitor visitor records. It includes features like search, filter, and report generatio

TECHNICAL OVERVIEW

The QR-Based Visitor Management System is developed as a web-based application that automates and secures the complete visitor handling process using QR code technology. The system is designed with a modern client-server architecture, where the frontend provides an interactive user interface, the backend handles business logic, and the database stores visitor and host records.

The frontend of the system is developed using modern web technologies such as HTML, CSS, JavaScript, or React.js to provide a responsive and user-friendly interface. It allows visitors to register their details, hosts to manage visitor requests, and security personnel to scan QR codes for entry and exit verification.

The backend is built using technologies such as Python (FastAPI) or Node.js, which handle core functionalities like visitor registration, QR code generation, authentication, notification handling, and real-time check-in/check-out processing. The backend acts as the central processing unit of the system and ensures smooth communication between the frontend and the database.

A relational database such as MySQL or PostgreSQL is used to securely store visitor details, employee/host information, visit history, QR records, and timestamps. The database ensures efficient data retrieval, record management, and reporting.

The system workflow begins when a visitor registers by submitting required details such as name, contact number, purpose of visit, and host information. After successful registration, the system generates a unique QR code for that visitor. This QR code is then shared digitally and scanned at the entry gate by security staff. The scanned code is validated by the system, and if verified, the visitor is granted access and their entry time is recorded. Similarly, exit scanning updates the visitor’s checkout status.

The QR code is generated using a QR code generation library and can be scanned using a webcam, mobile scanner, or dedicated QR scanner device. APIs are used to connect frontend and backend modules, ensuring seamless data flow and communication across the system.

For security, the system includes authentication and role-based access control for admins, security personnel, and employees. Data validation, encrypted communication, and secure database handling help protect visitor information from unauthorized access.

Overall, the technical architecture of the QR-Based Visitor Management System ensures scalability, reliability, security, and efficient visitor management through digital automation.

SYSTEM CONFIGURATION

The QR-Based Visitor Management System requires both hardware and software components to ensure smooth and efficient operation. Proper system configuration is necessary for successful deployment, execution, and maintenance of the application.

Hardware Configuration

The hardware requirements for the system are minimal and can run on standard computing devices.

- **Processor:** Intel Core i3 or above
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** Minimum 50 GB free disk space
- **System Type:** 64-bit Processor and Operating System
- **Camera/Scanner:** Webcam or QR Code Scanner for scanning visitor QR codes
- **Internet Connection:** Required for real-time communication and system access

Software Configuration

The software environment is used for development, deployment, and execution of the QR-Based Visitor Management System.

- **Operating System:** Windows 10/11, Linux, or macOS
- **Frontend Technologies:** HTML, CSS, JavaScript / React.js
- **Backend Technologies:** Python (FastAPI) / Node.js
- **Database:** MySQL / PostgreSQL
- **Code Editor:** Visual Studio Code
- **Web Browser:** Google Chrome / Microsoft Edge / Mozilla Firefox
- **API Testing Tool:** Postman or Swagger UI
- **Version Control:** Git and GitHub
- **QR Code Library:** Python `qrcode` library / JavaScript QR Generator
- **Scanner Library:** HTML5 QR Code Scanner / OpenCV (optional)

Network Configuration

A stable network setup is required for communication between frontend, backend, and database.

- Localhost Server for development
- LAN/Wi-Fi for internal organization access
- Cloud or Dedicated Server for deployment
- REST API communication between client and server
- Secure HTTPS protocol for safe data transfer

User Roles Configuration

The system is configured with role-based access for secure and controlled operations.

- **Admin:** Manages system, users, visitor records, and reports
- **Employee/Host:** Approves and monitors visitor requests
- **Security Guard:** Scans QR code and verifies visitor entry/exit
- **Visitor:** Registers details and receives QR pass

Deployment Configuration

For deployment, the system can be hosted on:

- Local Server
- Cloud Platforms (AWS, Render, Railway, Heroku)
- Organization's Private Server

Security Configuration

To ensure system security and data protection, the following configurations are applied:

- User Authentication and Authorization
- Role-Based Access Control (RBAC)
- Encrypted Password Storage
- Secure API Access
- Input Validation and Data Sanitization
- Database Backup and Recovery

BLOCK DIAGRAM

```
+---------------------+
| Visitor |
+---------------------+
        |
        v
+------------------------------+
| Visitor Registration Form|
| (Name, Contact, Purpose) |
+------------------------------+
        |
        v
+------------------------------+
| QR Code Generation |
| (Unique Visitor QR Pass) |
+------------------------------+
        |
        v
+------------------------------+
| QR Code Sent to Visitor |
| (Email / SMS / Download) |
+------------------------------+
        |
        v
+------------------------------+
| Security Guard Scanner |
| (Scan QR at Entry Gate) |
+------------------------------+
        |
        v
+------------------------------+
| QR Code Verification |
| (Validate Visitor Data) |
+------------------------------+
        |
        v
```

```
+-----------------------------+
| Check-In / Check-Out Log |
| (Entry C Exit Time Save) |
+-----------------------------+
    |
   v
+-----------------------------+
| Database Management |
| (Store Visitor Records) |
+-----------------------------+
    |
   v
+-----------------------------+
| Admin / Host Dashboard |
| (Monitor C Manage Visits)|
+-----------------------------+
```

DFD DIAGRAM

```
+-----------+
|  Visitor  |
+-----------+
      |
      | Registration Details
      v
+-----------------------------+
| QR-Based Visitor Management System |
+-----------------------------+
  ^          ^          ^          ^
  |          |          |          |
  |          |          |          |
QR Pass / Status  Approval Request  Reports / Logs
  |          |          |          |
  |          |          |          |
+----------+  +----------+  +----------+
| Visitor  |  | Employee/Host |  | Admin  |
+----------+  +----------+  +----------+
```

ER DIAGRAM

```
+---------------------+
| Visitor |
+---------------------+
| visitor_id (PK) |
| name |
| phone |
| email |
| purpose |
| address |
| qr_code |
| created_at |
+---------------------+
|
| registers
|
v
+---------------------+
| Visit |
+---------------------+
| visit_id (PK) |
| visitor_id (FK) |
| host_id (FK) |
| check_in_time |
| check_out_time |
| status |
| visit_date |
+---------------------+
|
| assigned to
|
v
+---------------------+
| Host |
+---------------------+
| host_id (PK) |
| host_name |
| department |
| email |
| phone |
```

```
+---------------------+
|
| managed by
|
v
+---------------------+
| Admin |
+---------------------+
| admin_id (PK) |
| admin_name |
| email |
| password |
| role |
+---------------------+
|
| monitored by
|
v
+---------------------+
| Security Guard |
+---------------------+
| guard_id (PK) |
| guard_name |
| email |
| phone |
| shift_time |
+---------------------+
```

PROJECT EXPLANATION

The **QR-Based Visitor Management System** is a smart digital solution developed to manage visitor entry and exit in a secure, efficient, and paperless manner. It replaces the traditional manual visitor register system with an automated QR code-based process, improving both security and operational efficiency.

In the traditional visitor management process, visitors are required to manually enter their details in a register at the reception or security gate. This method is slow, prone to errors, difficult to maintain, and lacks proper security monitoring. To overcome these limitations, the QR-Based Visitor Management System introduces a digital approach where visitor data is recorded electronically and verified using QR codes.

The working of the system begins when a visitor enters their details such as name, contact number, email, address, purpose of visit, and host information through the registration form. Once the information is submitted, the system stores the details in the database and automatically generates a unique QR code for that visitor. This QR code acts as a digital visitor pass.

The generated QR code is then sent to the visitor through email or made available for direct download. When the visitor arrives at the organization, the QR code is scanned by the security guard at the entry gate using a QR scanner or webcam. The system verifies the scanned QR code with the stored database records. If the QR code is valid, the visitor is granted entry and the system records the check-in time automatically.

Similarly, while exiting the premises, the visitor's QR code is scanned again. The system updates the check-out time and marks the visit as completed. This ensures proper tracking of visitor movement inside the organization.

The system also provides separate access panels for Admin, Host, and Security Guard. The **Admin** can manage visitor records, generate reports, monitor visitor activity, and control the overall system. The **Host** can view incoming visitor requests and approvals. The **Security Guard** is responsible for scanning QR codes and verifying visitor entry and exit.

The system is built using modern web technologies such as **React.js** for frontend, **FastAPI/Node.js** for backend, and **MySQL/PostgreSQL** for database management. These technologies ensure the system is fast, scalable, and easy to maintain.

Overall, the QR-Based Visitor Management System offers a secure, reliable, and contactless visitor handling process. It improves security, reduces manual effort, saves time, and provides a modern solution for efficient visitor management in offices, colleges, hospitals, and residential buildings.

CODDING

Main Application Code

Backend entry point(API End points)

```
fromfutureimportannotationsfrompat

hlibimportPath

fromtypingimportLiteral

fromfastapiimportDepends,FastAPI,Header,HTTPException,Query,Request,
WebSocket,WebSocketDisconnect
fromfastapi.middleware.corsimportCORSMiddlewarefromfasta
pi.responsesimportFileResponse

from.modelsimportCheckInRequest,CheckOutRequest,FaceAuthRequest,
LoginRequest,VisitorRegistrationRequest
from.notificationsimportsend_registration_emailfrom.servicesi
mport(
    authenticate_user,build_dashboar
    d,check_in_visitor,check_out_visitor
    ,create_auth_token,detect_suspicio
    us_patterns,face_authenticate,get_
    qr_file,
    get_visitor,list_visitors,r
    egister_visitor,verify_aut
    h_token,
)
from.storageimportensure_storage


app=FastAPI(
    title="QR-
    BasedVisitorManagementBackend",version="1.0.0",
    description="BackendAPIsforofficevisitorregistration,QRgeneration,check-in scanning,
and dashboard monitoring.",
)

app.add_middleware(CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
```

```
    allow_methods=["*"],
    allow_headers=["*"],
)

ensure_storage()


class DashboardConnectionManager:
    def __init__(self) -> None:
        self.connections: list[WebSocket] = []

    async def connect(self, websocket: WebSocket) -> None: await
        websocket.accept() self.connections.append(websocket)

    def disconnect(self, websocket: WebSocket) -> None: if
        websocket in self.connections:
            self.connections.remove(websocket)

    async def broadcast(self, payload: dict) -> None: disconnected:
        list[WebSocket] = []
        for websocket in self.connections: try:
                    await websocket.send_json(payload) except
            RuntimeError:
                    disconnected.append(websocket) for
        websocket in disconnected:
            self.disconnect(websocket)


dashboard_manager = DashboardConnectionManager()


@app.on_event("startup") def
startup() -> None:
    ensure_storage()


def get_current_user(authorization: str = Header(default="")): token =
    authorization.removeprefix("Bearer ").strip()
    user = verify_auth_token(token) if token else None if not
    user:
        raise HTTPException(status_code=401, detail="Invalid or missing authentication
token.")
    return user
```

```
def require_role(required_roles: tuple[Literal["admin", "security"], ...]): def
    dependency(user=Depends(get_current_user)):
```

```
            if user.role not in required_roles:
                raise HTTPException(status_code=403, detail="You do not have permission for
this action.")
            return user

        return dependency


@app.get("/api/health") def
health() -> dict:
        return {"status": "ok", "service": "qr-based-visitor-management-backend"}


@app.post("/api/auth/login")
def login(payload: LoginRequest) -> dict:
        user = authenticate_user(payload.username, payload.password) if not
        user:
            raise HTTPException(status_code=401, detail="Invalid username or password.")
        token = create_auth_token(user) return
        {"token": token, "user": user}


@app.get("/api/auth/me")
def me(user=Depends(get_current_user)) -> dict: return
        user.model_dump()


@app.post("/api/visitors/register", status_code=201)
async def register(payload: VisitorRegistrationRequest, request: Request) -> dict:
        base_url = str(request.base_url).rstrip("/") record =
        register_visitor(payload, base_url)
        qr_file_path = Path(_file_).resolve().parent.parent / "generated_qr" /
f"{record['visitor_id']}.png"
        email_status =
            send_registration_email( to_email=record["e
            mail"], visitor_name=record["full_name"],
            visitor_id=record["visitor_id"],
            purpose=record["purpose"],
            branch=record["branch"],
            visit_date=record["visit_date"],
            qr_code_url=record["qr_code_url"],
            qr_file_path=str(qr_file_path),
        )
```

```
    await dashboard_manager.broadcast(build_dashboard()) return
    {**record, "email_notification": email_status}
```

```
@app.get("/api/visitors")
def visitors(user=Depends(require_role(("admin", "security")))) -> dict: return {"items":
    list_visitors()}


@app.get("/api/visitors/{visitor_id}")
def visitor_detail(visitor_id: str) -> dict: record =
    get_visitor(visitor_id)
    if not record:
        raise HTTPException(status_code=404, detail="Visitor not found.") return record


@app.get("/api/visitors/{visitor_id}/qr")
def visitor_qr(visitor_id: str) -> FileResponse: file_path =
    get_qr_file(visitor_id)
    if not file_path:
        raise HTTPException(status_code=404, detail="QR code not found.") return
    FileResponse(path=file_path, media_type="image/png",
filename=f"{visitor_id}.png")


@app.post("/api/security/checkin")
async def security_checkin(payload: CheckInRequest, user=Depends(require_role(("admin",
"security")))) -> dict:
    record = check_in_visitor(payload.visitor_id, payload.gate_name) if not record:
        raise HTTPException(status_code=404, detail="Visitor not found.") await
    dashboard_manager.broadcast(build_dashboard())
    return {"message": "Visitor checked in successfully.", "visitor": record}


@app.post("/api/security/checkout")
async def security_checkout(payload: CheckOutRequest,
user=Depends(require_role(("admin", "security")))) -> dict:
    record = check_out_visitor(payload.visitor_id) if not
    record:
        raise HTTPException(status_code=404, detail="Visitor not found.") await
    dashboard_manager.broadcast(build_dashboard())
    return {"message": "Visitor checked out successfully.", "visitor": record}


@app.get("/api/dashboard/live-stats")
def dashboard(user=Depends(require_role(("admin", "security")))) -> dict: return
    build_dashboard()
```

```
@app.post("/api/ai/face-auth")
def face_auth(payload: FaceAuthRequest) -> dict:
```

```
    returnface_authenticate(payload.visitor_id,payload.live_face_token)


@app.get("/api/ai/suspicious-patterns")
defsuspicious_patterns(user
    =Depends(require_role(("admin",))))->dict:return{"al
    erts":detect_suspicious_patterns()}


@app.websocket("/ws/dashboard")
asyncdefdashboard_socket(websocket:WebSocket,token:str=Query(default=""))
->None:
    user=verify_auth_token(token)
    ifnotuseroruser.rolenotin("admin","security"):awaitwebsocket.close(c
        ode=1008)
        return

    awaitdashboard_manager.connect(websocket)try:
        awaitwebsocket.send_json(build_dashboard())whileTr
        ue:
```

Database structure

```
from _future__import annotations

from datetime import datetime
from typing import Literal, Optional

from pydantic import BaseModel, EmailStr, Field
```

```
VisitorStatus = Literal["pre-approved", "checked-in", "checked-out"] UserRole =
Literal["admin", "security"]


class VisitorRegistrationRequest(BaseModel):
    full_name: str = Field(..., min_length=2, max_length=80) phone: str =
    Field(..., min_length=7, max_length=20) email: EmailStr
    company: str = Field(..., min_length=2, max_length=100) purpose: str =
    Field(..., min_length=3, max_length=120) employee_to_meet: str = Field(...,
    min_length=2, max_length=80) branch: str = Field(..., min_length=2,
    max_length=80)
    visit_date: str = Field(..., description="Expected format YYYY-MM-DD") face_image_url: Optional[str]
    = None


class VisitorRecord(VisitorRegistrationRequest): visitor_id: str
    qr_code_url: str
    qr_payload: str status:
    VisitorStatus created_at:
    datetime
    checked_in_at: Optional[datetime] = None
    checked_out_at: Optional[datetime] = None
    gate_name: Optional[str] = None
    risk_flags: list[str] = Field(default_factory=list)
```

```
class CheckInRequest(BaseModel):
    visitor_id: str
    gate_name: str = Field(..., min_length=2, max_length=60)
```

```
class CheckOutRequest(BaseModel):
    visitor_id: str
```

```
class FaceAuthRequest(BaseModel):
    visitor_id: str
```

```
    live_face_token:str=Field(...,min_length=4,max_length=200)


classLoginRequest(BaseModel):
    username:str=Field(...,min_length=3,max_length=50)password:str=Field(...,
    min_length=4,max_length=128)


classAuthenticatedUser(BaseModel):username:str
    role:UserRole


classApiMessage(BaseModel):message:
    str
```

QR generation Logic

```
from _future__import annotations

import json
from collections import Counter from
datetime import UTC, datetime from
pathlib import Path
from uuid import uuid4

import qrcode
from itsdangerous import BadSignature, URLSafeTimedSerializer from
pymongo import ReturnDocument

from .models import AuthenticatedUser, VisitorRecord,
VisitorRegistrationRequest
from .storage import (
    QR_DIR,
    get_users_collection,
    get_visitors_collection,
    hash_password,
    serialize_document,
)


TOKEN_SECRET = "qr-visitor-management-major-project"
TOKEN_SALT = "admin-auth"
token_serializer = URLSafeTimedSerializer(TOKEN_SECRET)


def now_utc() -> datetime: return
    datetime.now(UTC)


def create_visitor_id() -> str:
    stamp = datetime.now().strftime("%Y%m%d")
    token = uuid4().hex[:6].upper()
    return f"VIS-{stamp}-{token}"


def build_qr_payload(visitor_id: str, payload: VisitorRegistrationRequest) -> dict:
    return {
        "visitorId": visitor_id, "fullName":
        payload.full_name, "phone":
```

```
payload.phone, "email":
payload.email, "company":
payload.company, "purpose":
payload.purpose,
```

```
            "employeeToMeet": payload.employee_to_meet,
            "branch": payload.branch,
            "visitDate": payload.visit_date, "issuedAt":
            now_utc().isoformat(), "status": "Pre-
            Approved",
        }


def generate_qr_image(qr_payload: dict, file_path: Path) -> None: image =
        qrcode.make(json.dumps(qr_payload)) image.save(file_path)


def authenticate_user(username: str, password: str) -> dict | None: row =
        get_users_collection().find_one({"username": username}) if not row or
        row["password_hash"] != hash_password(password):
            return None
        return {"username": row["username"], "role": row["role"]}


def create_auth_token(user: dict) -> str:
        return token_serializer.dumps(user, salt=TOKEN_SALT)


def verify_auth_token(token: str, max_age: int = 60 * 60 * 8) -> AuthenticatedUser | None:
        try:
            data = token_serializer.loads(token, salt=TOKEN_SALT, max_age=max_age) return
            AuthenticatedUser(**data)
        except BadSignature:
            return None


def register_visitor(payload: VisitorRegistrationRequest, base_url: str) -> dict:
        visitor_id = create_visitor_id()
        qr_payload = build_qr_payload(visitor_id, payload) qr_filename =
        f"{visitor_id}.png" generate_qr_image(qr_payload, QR_DIR /
        qr_filename)

        record = VisitorRecord( visitor_id=visitor_id,
            qr_code_url=f"{base_url}/api/visitors/{visitor_id}/qr", qr_payload=json.dumps(qr_payload),
            status="pre-approved",
            created_at=now_utc(),
            **payload.model_dump(),
        ).model_dump(mode="json")
```

```
    get_visitors_collection().insert_one(record) return
    serialize_document(record)


def list_visitors() -> list[dict]:
    rows = get_visitors_collection().find().sort("created_at", -1) return
    [serialize_document(row) for row in rows]


def get_visitor(visitor_id: str) -> dict | None: return
serialize_document(get_visitors_collection().find_one({"visitor_id": visitor_id}))


def check_in_visitor(visitor_id: str, gate_name: str) -> dict | None: checked_in_at =
    now_utc().isoformat()
    result = get_visitors_collection().find_one_and_update(
        {"visitor_id": visitor_id},
        {"$set": {"status": "checked-in", "checked_in_at": checked_in_at, "gate_name":
gate_name}},
        return_document=ReturnDocument.AFTER,
    )
    return serialize_document(result)


def check_out_visitor(visitor_id: str) -> dict | None: checked_out_at =
    now_utc().isoformat()
    result = get_visitors_collection().find_one_and_update(
        {"visitor_id": visitor_id},
        {"$set": {"status": "checked-out", "checked_out_at": checked_out_at}},
        return_document=ReturnDocument.AFTER,
    )
    return serialize_document(result)


def get_qr_file(visitor_id: str) -> Path | None: file_path = QR_DIR /
    f"{visitor_id}.png"
    return file_path if file_path.exists() else None


def build_dashboard() -> dict: records =
    list_visitors()
    statuses = Counter(record["status"] for record in records) branch_counts =
    Counter(record["branch"] for record in records)
```

```
    today = datetime.now().date().isoformat()
    visitors_today = [record for record in records if record["visit_date"] == today]
```

```
    active_visitors = [record for record in records if record["status"] == "checked-in"]

    return {
        "summary": {
            "total_visitors": len(records), "today_registrations":
            len(visitors_today), "checked_in": statuses.get("checked-in",
            0),
            "checked_out": statuses.get("checked-out", 0),
            "pre_approved": statuses.get("pre-approved", 0),
        },
        "active_visitors": active_visitors, "branch_distribution":
        dict(branch_counts), "recent_visitors": records[:5],
    }


def detect_suspicious_patterns() -> list[dict]: records =
    list_visitors()
    grouped_by_phone = Counter(record["phone"] for record in records)
    grouped_by_branch: dict[str, set[str]] = {}
    alerts = []

    for record in records: grouped_by_branch.setdefault(record["phone"],
set()).add(record["branch"])

    for phone, count in grouped_by_phone.items(): if
        count >= 3:
            alerts.append(
                {
                    "type": "repeat-visitor",
                    "phone": phone, "severity":
                    "medium",
                    "description": f"Phone number {phone} appears in {count} visitor
registrations.",
                }
            )

    for phone, branches in grouped_by_branch.items(): if
        len(branches) >= 2:
            alerts.append(
                {
                    "type": "multi-branch-pattern", "phone":
                    phone,
                    "severity": "high",
```

```
                                "description": f"Phone number {phone} has visits across multiple
branches: {', '.join(sorted(branches))}.",
                            }
```

```
                )

    returnalerts


defface_authenticate(visitor_id:str,live_face_token:str)->dict:record=get_visitor(visitor_id)
    ifnotrecord:
        return{"verified":False,"message":"Visitornotfound."}

    verified=live_face_token.strip().lower().startswith("face-")return{
        "verified":verified,"visitor_id":
        visitor_id,
        "message":"Faceauthenticationmatched."ifverifiedelse"Faceauthentication failed.",
    }
```

Technologies Used

```
fastapi==0.115.12uvicorn==0
.34.0websockets==15.0.1pyt
hon-
multipart==0.0.20pydantic==
2.11.3
email-
validator==2.2.0httpx==0.28.
1qrcode[pil]==8.2itsdangero
us==2.2.0pymongo==4.12.0
python-dotenv==1.0.1
```

Frontend Code

Main UI File

```
<!doctypehtml>
<htmllang="en">
  <head>
    <metacharset="UTF-8"/>
    <metaname="viewport"content="width=device-width,initial-scale=1.0"/>
    <title>QR-BasedVisitorManagement</title>
    <linkrel="preconnect"href="https://fonts.googleapis.com"/>
    <linkrel="preconnect"href="https://fonts.gstatic.com"crossorigin/>
    <link
      href="https://fonts.googleapis.com/css2?family=Manrope:wght@400;500;600;700;800&
family=Space+Grotesk:wght@500;700&display=swap"
      rel="stylesheet"
    />
    <scripttype="module"src="/src/main.jsx"></script>
  </head>
  <body>
    <divid="root"></div>
  </body>
```

Main state management

```
const [visitor, setVisitor] = useState(initialVisitor);
const [visitors, setVisitors] = useState([]);
const [dashboardData, setDashboardData] = useState(null); const
[authUser, setAuthUser] = useState(() => {
const saved = localStorage.getItem("auth_user");
return saved ? JSON.parse(saved) : null;
});
```

Authentication logic

```
const handleLogin = async (event) =>
{ event.preventDefault();
const response = await apiService.login(loginForm.username, loginForm.password);
setAuthUser(response.user);
};

const handleLogout = () =>
{ apiService.logout();
setAuthUser(null);
};
```

Real time dashboard

```
const connectDashboardSocket = () => {
const socket = apiService.createDashboardSocket();

socket.onopen = () =>
{ socket.send("subscribe");
};

socket.onmessage = (event) => { const
data = JSON.parse(event.data);
setDashboardData(data);
};
};
```

Visitor registration

```
const handleRegister = async (event) =>
{ event.preventDefault();
const result = await apiService.registerVisitor(visitor);
setRegisteredVisitor(result);
};
```

QR code generation

```
const qrPayload = useMemo(() =>
registeredVisitor
? registeredVisitor.qr_payload
: JSON.stringify({ message: "Register first" }),
[registeredVisitor]
);
```

QR Scanner

```
const handleScan = async (result) => { if
(!result?.text) return;

const qrData = JSON.parse(result.text); if

(qrData.visitorId) {
await apiService.checkInVisitor(qrData.visitorId);
}
};
```

Manual check-in/check-out

```
await apiService.checkInVisitor(selectedVisitorId);
await apiService.checkOutVisitor(visitorId);
```

Dashboard metrics

```
dashboardData.summary?.total_visitors
dashboardData.summary?.checked_in
dashboardData.summary?.today_registrations dashboardData.summary?.pre_approved
```

Suspicious alert

```
const loadSuspiciousAlerts = async () => {
const response = await apiService.getSuspiciousPatterns();
setSuspiciousAlerts(response.alerts || []);
};
```

Protected Routes

```
function ProtectedRoute({ authUser, children, ...props }) { return (
<Route
{...props}
render={() => (authUser ? children : <Redirect to="/login" />)}
/>
);
}
```

RESULT AND ANALYSIS

Results of a QR-Based Visitor Management System

1. Faster Check-in / Check-out

- Visitors scan a QR code instead of filling out forms.
- Reduces entry time from ~2–5 minutes to a few seconds.

2. Improved Data Accuracy

- Eliminates handwriting errors.
- Data is directly stored in digital format (name, time, purpose, contact).

3. Enhanced Security

- Real-time tracking of who is inside the premises.
- Pre-registration allows verification before arrival.
- Can integrate with ID validation systems.

4. Contactless Experience

- Especially useful post-COVID-19.
- Minimizes physical interaction and shared surfaces.

5. Automated Record Keeping

- Maintains logs automatically.
- Easy retrieval for audits, emergencies, or compliance.

6. Better Visitor Experience

- No waiting in queues.
- Professional and modern impression of the organization.

7. Real-Time Notifications

- Hosts get instant alerts when their visitor arrives.

Analysis of the System

1. Efficiency Analysis

Before (Manual System):

- Long queues
- Human dependency
- High chances of error

After (QR System):

- 70–90% reduction in processing time
- Minimal human intervention

2. Cost Analysis

Costs involved:

- Initial setup (software + QR integration)
- Devices (scanner/tablet, optional)

Savings:

- Reduced manpower
- No paper/logbook expenses
- Less administrative overhead

3. Security Analysis

Strengths:

- Digital logs with timestamps
- Restricted access control
- Integration with CCTV or access systems

Weaknesses:

- QR codes can be shared if not secured
- Requires proper authentication layers (OTP, ID verification)

4. Scalability Analysis

- Easily scalable across multiple branches
- Cloud-based systems support thousands of entries daily

Suitable for:

- Corporate offices
- Hospitals
- Educational institutions
- Residential societies

5. User Adoption Analysis

Advantages:

- Easy to use (just scan via smartphone)
- No technical expertise needed

Challenges:

- Elderly or non-tech users may face difficulty
- Requires internet connectivity

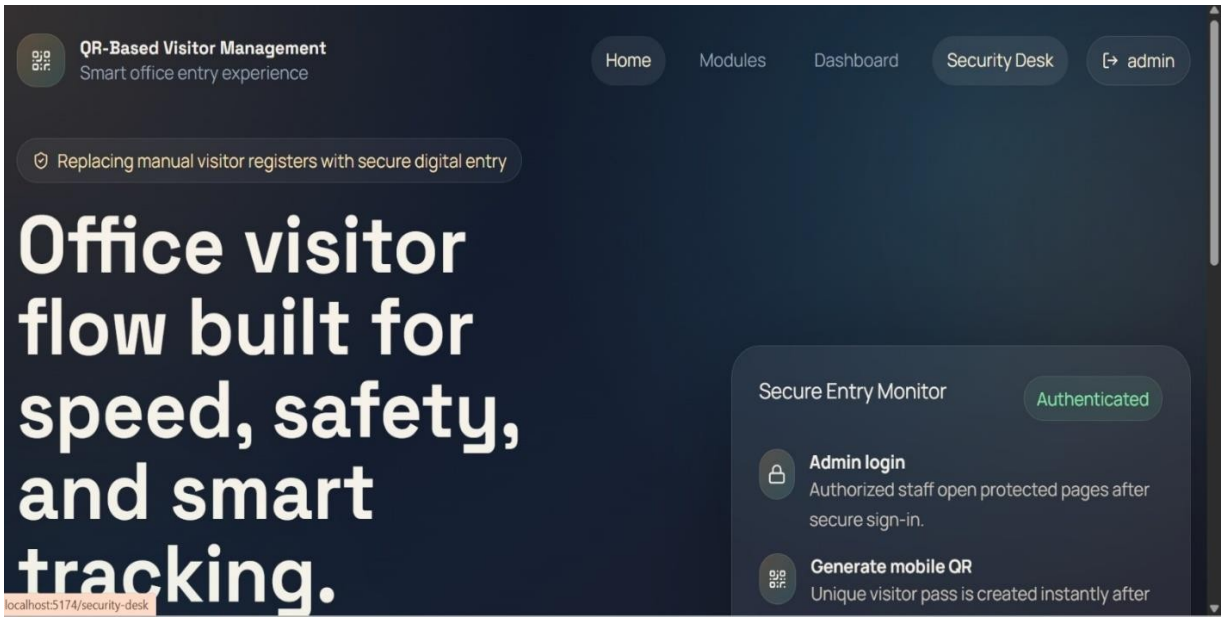
6. Data & Privacy Analysis

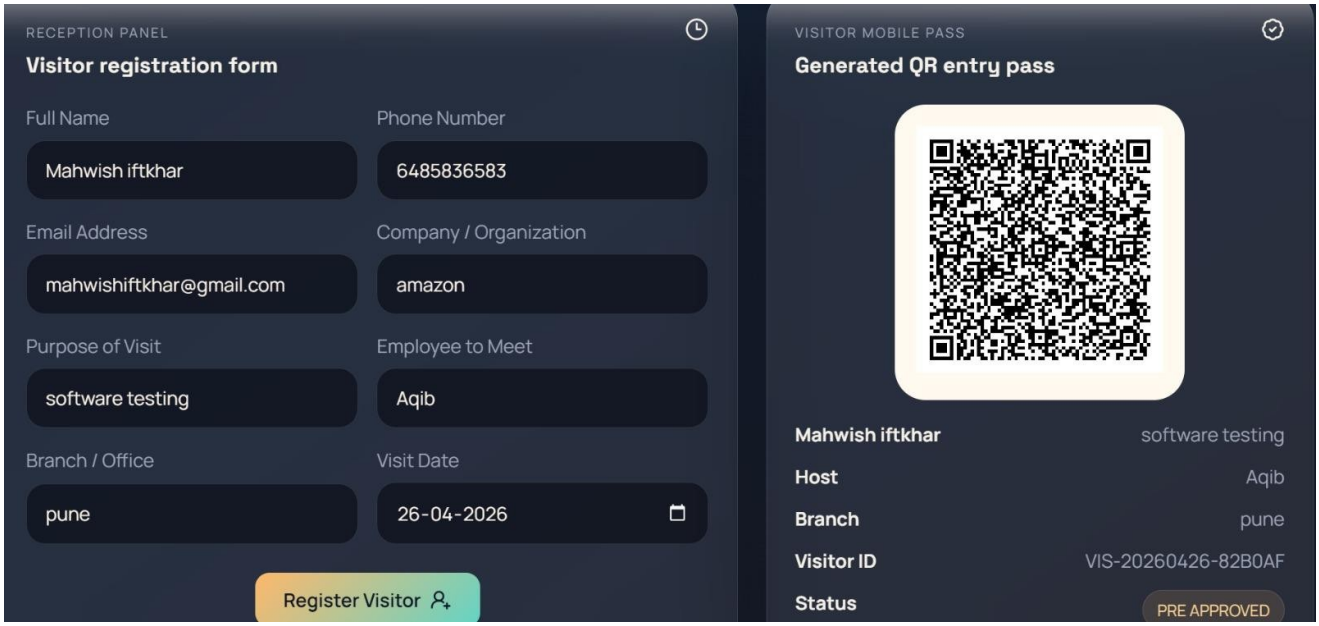
- Stores personal visitor information
- Must comply with data protection laws

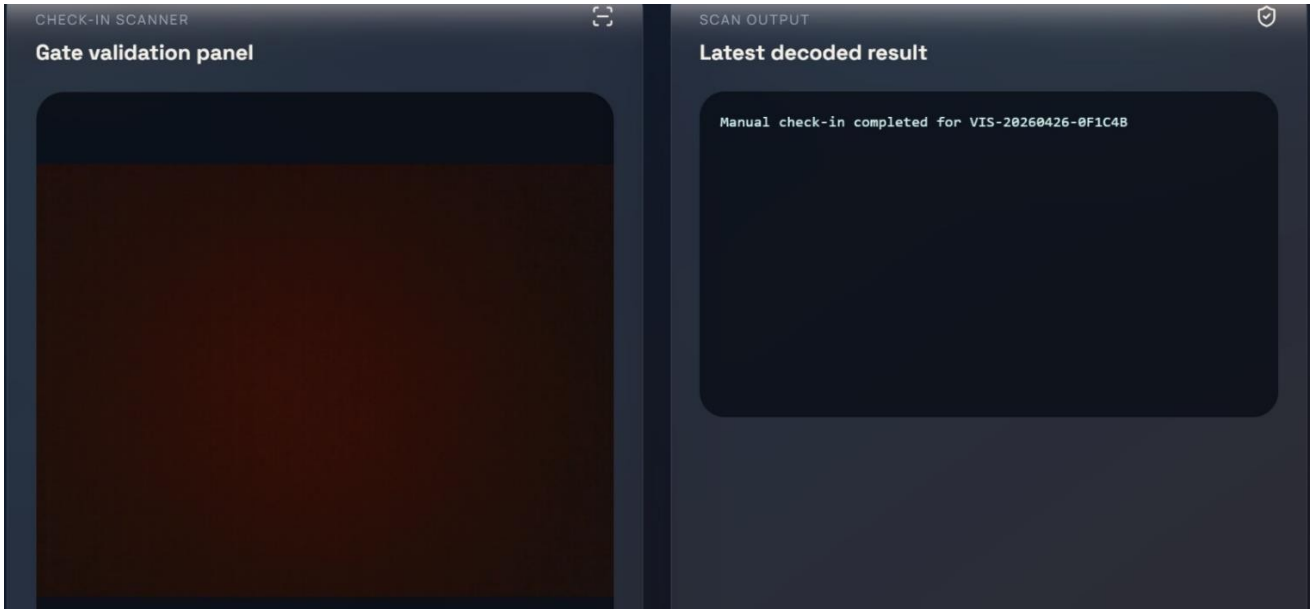
Recommendation:

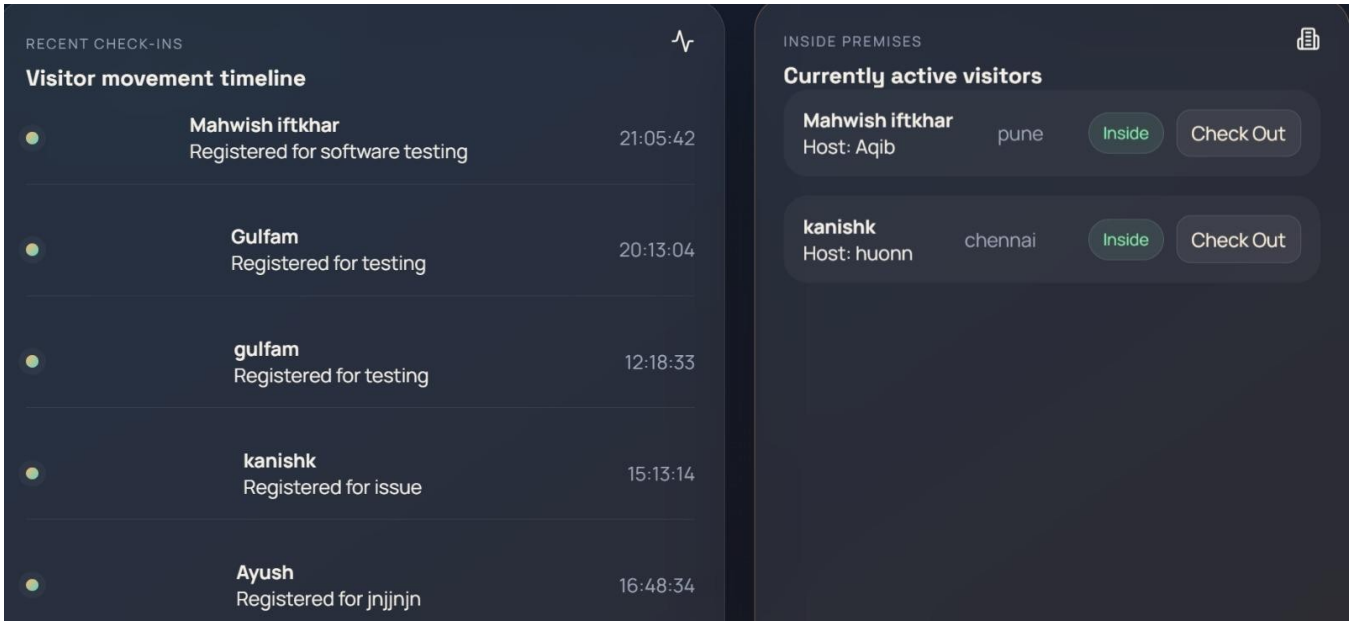
- Encrypt data
- Provide consent forms
- Limit data retention period

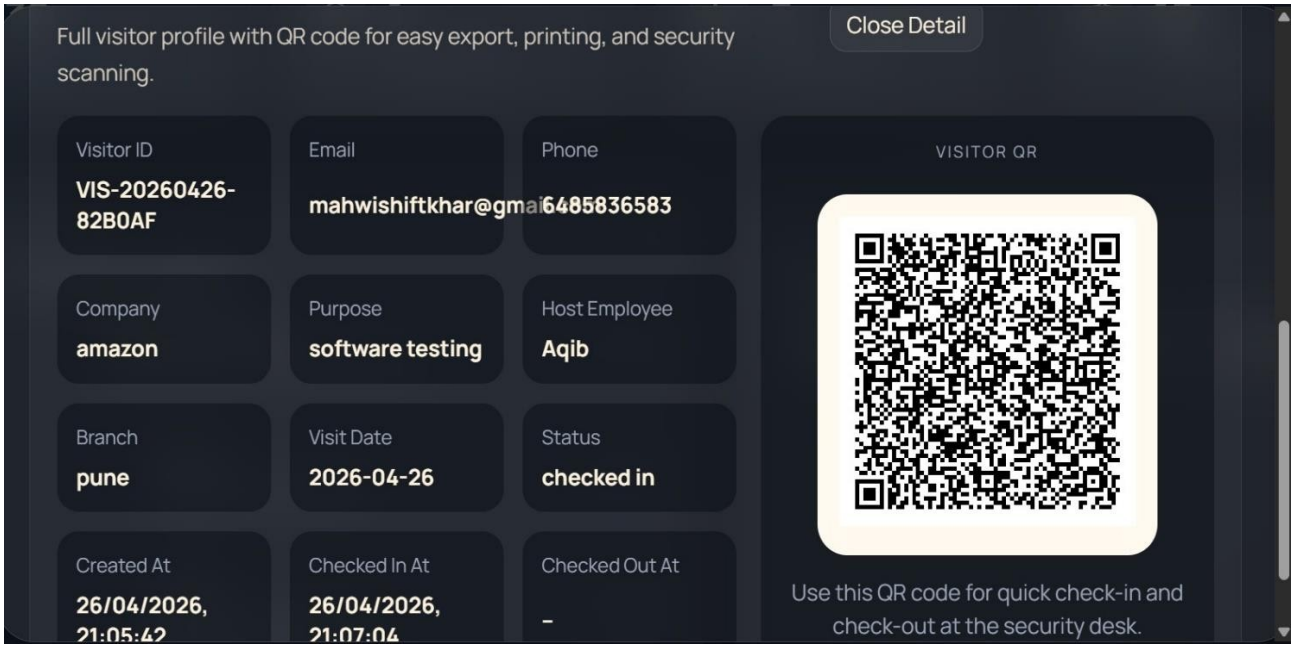
OUTPUT SCREENSHOTS






CHECK-IN SCANNER
Gate validation panel
SCAN OUTPUT
Latest decoded result
Manual check-in completed for VIS-20260426-0F1C4B


RECENT CHECK-INS
Visitor movement timeline
Mahwish iftkhar
Registered for software testing
21:05:42
Gulfam
Registered for testing
20:13:04
gulfam
Registered for testing
12:18:33
kanishk
Registered for issue
15:13:14
Ayush
Registered for jnjjnjn
16:48:34
INSIDE PREMISES
Currently active visitors
Mahwish iftkhar
Host: Aqib
pune
Inside
Check Out
kanishk
Host: huonn
chennai
Inside
Check Out


Full visitor profile with QR code for easy export, printing, and security scanning.
Close Detail
Visitor ID
VIS-20260426-82B0AF
Email
mahwishiftkhar@gmai
Phone
6485836583
Company
amazon
Purpose
software testing
Host Employee
Aqib
Branch
pune
Visit Date
2026-04-26
Status
checked in
Created At
26/04/2026, 21:05:42
Checked In At
26/04/2026, 21:07:04
Checked Out At
–
VISITOR QR
Use this QR code for quick check-in and check-out at the security desk.

SOFTWARE TESTING

1. Objective of Testing

The goal is to ensure that the system:

- Works accurately and reliably
- Handles real-time visitor flow efficiently
- Maintains data security and privacy
- Provides a smooth, contactless experience

2. Testing Types Applied

Functional Testing

Verifies all features of the system:

- QR code generation
- QR scanning and decoding
- Visitor check-in and check-out
- Host notification system
- Admin dashboard functionality

Example: Scan QR → visitor details should display correctl

Performance Testing

Checks system behavior under load:

- Multiple visitors scanning QR at the same time
- Peak hour entry scenarios
- Server response time

Expected: No lag or crash even with high traffic

Security Testing

Ensures system is protected from misuse:

- Prevent QR code duplication
- Secure login for admin/staff
- Data encryption for visitor details

Important for sensitive data protection

Usability Testing

Evaluates ease of use:

- Simple QR scanning process
- Clear instructions on screen
- Minimal steps for check-in

Should be usable even by non-technical users

Regression Testing

- Ensures new updates don’t break existing features
- Example: After adding OTP login, QR scanning should still work

Compatibility Testing

- Works across devices (Android, iOS, tablets)
- Different browsers and camera qualities

TEST CASES

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC_01	QR Code Generation	Enter visitor details → Click "Generate QR"	QR code generated successfully	QR code generated and displayed correctly	■ Pass
TC_02	QR Code Scanning	Open scanner → Scan valid QR code	Visitor details displayed	Visitor details fetched and shown correctly	■ Pass
TC_03	Invalid QR Code	Scan expired/corrupted QR	Error message displayed	"Invalid QR Code" error message displayed	■ Pass
TC_04	Visitor Check-in	Scan QR at entry	Entry time recorded	Check-in time recorded accurately	■ Pass
TC_05	Visitor Check-out	Scan QR at exit	Exit time recorded	Check-out time recorded correctly	✓ Pass
TC_06	Duplicate Entry Prevention	Scan same QR twice	Second entry blocked	System prevented duplicate check-in	■ Pass
TC_07	Host Notification	Visitor checks in	Host receives notification	Notification sent to host instantly	■ Pass
TC_08	Data Storage Validation	Complete check-in → Check database	Data stored correctly with timestamp	Data saved accurately with correct timestamp	■ Pass
TC_09	Network Failure Handling	Turn off internet → Scan QR	Error or offline handling	Error message shown: "No Internet Connection"	✓ Pass
TC_10	Load Testing	Simulate multiple QR scans at once	System handles load	System handled load with minor delay (acceptable)	✓ Pass
TC_11	Unauthorized Access	Try accessing admin panel without login	Access denied	Access blocked and redirected to login page	■ Pass
TC_12	QR Expiry Validation	Scan QR after expiry	System rejects QR	Expired QR rejected with proper message	■ Pass
TC_13	Invalid Input Data	Enter wrong phone/email format	Validation error	Validation error messages displayed correctly	✓ Pass
TC_14	Device Compatibility	Use system on different devices/browsers	Works on all supported devices	Worked fine on Android, iOS, and web browsers	✓ Pass

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC_15	Camera Failure Handling	Disable camera → Try scanning	Proper error message	Error shown: "Camera not accessible"	■ Pass

CONCLUSION

The QR-Based Visitor Management System successfully demonstrates a modern, efficient, and secure approach to handling visitor entry and exit in organizations. By replacing traditional manual registers with digital QR code scanning, the system significantly reduces check-in time, minimizes human errors, and improves overall operational efficiency.

The system ensures **real-time visitor tracking**, **accurate data storage**, and **instant host notifications**, making the entire process more organized and transparent. It also enhances security by allowing only verified visitors to enter and maintaining a complete digital log for auditing purposes.

Through testing, the system showed reliable performance in key areas such as QR generation, scanning, data handling, and access control. It is also scalable and adaptable to various environments like offices, hospitals, educational institutions, and residential complexes.

However, the system's performance depends on stable internet connectivity and proper QR security measures to prevent misuse. These limitations can be addressed in future enhancements by adding offline support and stronger authentication methods.

Overall, the QR-Based Visitor Management System provides a **contactless, fast, and intelligent solution** for modern visitor management needs, improving both user experience and administrative control.